AI-BASED ADAPTIVE CRUISE CONTROL USING REINFORCEMENT LEARNING IN MATLAB

a project report submitted by

STEVE LEMUEL S (URK23RA2001)
ASWIN B (URK23RA2007)

in partial fulfillment for the award of the degree

of

B.TECH ROBOTICS AND AUTOMATION

In

DIVISION OF ROBOTICS ENGINEERING

under the supervision of

Dr. KINGSTON STANLEY P



DIVISION OF ROBOTICS ENGINEERING

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES

(Deemed-to-be-University)

Karunya Nagar, Coimbatore - 641 114. INDIA

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled

"AI-BASED ADAPTIVE CRUISE CONTROL USING REINFORCEMENT LEARNING IN MATLAB"

is the bonafide work of

STEVE LEMUEL S (URK23RA2001)
ASWIN B (URK23RA2007)

who carried out the project work under my supervision.

SIGNATURE

Dr. P. Rajalakshmy

Associate Professor
Division of Robotics Engineering

SIGNATURE

Dr. P. Kingston Stanley

Assistant Professor
Division of Robotics Engineering

Submitted for the Viva Voce held on …….

Internal Examiner

ACKNOWLEDGEMENT

First and foremost, we would like to thank Almighty God for all the blessings He has bestowed upon us to work thus far and finish this project. We are grateful to our most respected Founder **(late) Dr. D.G.S. Dhinakaran**, **C.A.I.I.B, Ph.D**., and Honorable Chancellor **Dr. Paul Dhinakaran, M.B.A, Ph.D**., for their grace and blessing.

We express our gratitude to the **Vice Chancellor Dr. R. Elijah Blessing and the Registrar Dr. S. J. Vijay, Ph.D.,** Karunya University, for their enduring leadership.

We extend thanks to our **Dean, School of Engineering and Technology**, **Dr. D. Nirmal**, Karunya University, for his excellent encouragements in course of this work.

We are very thankful to **Dr. P. Rajalakshmy, Associate Professor & Head, Department of Robotics Engineering** for providing her constant readiness in providing help and encouragement at all stages in our work.

Our sincere and special thanks to the course faculty, **Dr. Subathra**, **Professor**, for the immense help and guidance. We would like to extend a thankful heart for the constant support through the entire project.

We take this opportunity to thank our **Mentor Dr. L.D. Vijay Anand, Assistant Professor**, and **Dr. P. Kingston Stanley, Assistant Professor**, who had been always there for us.

We would like to convey gratitude to our Parents whose prayers and blessings were always there with us. Last but not the least, we would like to thank our friends and others who directly or indirectly helped in successful completion of this work.

ABSTRACT

Adaptive Cruise Control (ACC) automatically adjusts a vehicle's speed to keep a safe distance from the car in front, but most systems rely on **fixed, hand-tuned rules** that can react too late in unusual situations. This project presents an **AI-based ACC that learns to drive by itself using Reinforcement Learning**. A **Double Deep Q-Network (Double DQN)** agent observes six inputs, including the gap, the relative speed and the lead car's acceleration, and selects one of ten pedal levels, earning rewards for safe, steady and smooth driving. The entire system is **built from scratch in plain MATLAB**, without any pretrained model or reinforcement learning toolbox.

After 800 training drives, the AI recorded **zero collisions on 105 unseen test drives** and matched a tuned conventional controller on speed tracking. In a 60-drive hard-stop stress test, it recorded **zero collisions against eight** for the conventional controller. An interactive **3D live demo** exposed a hidden weakness that was then fixed. Future work includes continuous-action methods such as DDPG or TD3 for smoother pedal control.

TABLE OF CONTENTS

	CHAPTERS	PG NO.
	BONAFIDE CERTIFICATE	**2**
	ACKNOWLEDGEMENT	**3**
	ABSTRACT	**4**
	TABLE OF CONTENTS	**5**
	TABLE OF FIGURES	**6**
1	**INTRODUCTION**	**7**
2	**DESCRIPTION OF THE PROJECT**	**9**
3	**METHODOLOGY**	**10**
	3.1 SOFTWARE AND TOOLS USED	10
	3.2 SYSTEM ARCHITECTURE	10
	3.3 SIMULATED ENVIRONMENT	11
	3.4 STATE AND ACTIONS	12
	3.5 THE NEURAL NETWORK	13
	3.6 REWARD DESIGN	14
	3.7 CLASSIC BASELINE CONTROLLER	15
	3.8 DATA SETS FOR TRAINING AND TESTING	16
4	**ALGORITHM**	**17**
5	**USE CASES OF AI-BASED ACC AND WORKING PROCESS**	**20**
	5.1 USE CASES	20
	5.2 WORKING PROCESS	21
	5.3 TRAINING RESULT	22
	5.4 TEST RESULTS	23
	5.5 HARD-STOP SAFETY TEST	25
6	**FEATURES OF THIS PROJECT**	**27**
	6.1 SELF-LEARNING DRIVER	27
	6.2 BUILT FROM SCRATCH	27
	6.3 LEAD-CAR AWARENESS	27
	6.4 FAIR AND HONEST TESTING	27
	6.5 INTERACTIVE 3D LIVE DEMO	27
	6.6 EMERGENCY BRAKING SAFETY NET	27
7	**CONCLUSION**	**28**
8	**REFERENCES**	**29**

TABLE OF FIGURES

FIGURE NO.	DESCRIPTION	PG NO.
1.1	Ordinary Cruise Control and Adaptive Cruise Control	7
2.1	Overall Structure of the Project	9
3.1	Block Diagram of the System	11
3.2	Road, Safe Distance and Pedal Lag	12
3.3	The Six Inputs and the Ten Pedal Levels	13
3.4	The Neural Network	14
3.5	Marks Against the Gap to the Lead Car	15
3.6	Structure of the Classic Controller	16
3.7	The Four Sets of Drives	16
4.1	Training Flowchart	17
4.2	The Double DQN Learning Step	18
5.1	How the Trained Model Connects to the GUI	21
5.2	Layout of the Live Demo Window	22
5.3	Training Curve	23
5.4	AI and Classic ACC on a Hard-Brake Test Drive	24
5.5	Results on 105 Unseen Drives	24
5.6	Collisions in the Hard-Stop Test	26
6.1	Key Features at a Glance	27
7.1	Final Product: The Live 3D Demo	28

CHAPTER 1
INTRODUCTION

Modern road vehicles are increasingly equipped with **driver-assistance systems** that make driving safer and less tiring. One of the most widely used of these is **Adaptive Cruise Control (ACC)**. An ordinary cruise control only holds the speed chosen by the driver. Adaptive cruise control goes one step further: a radar measures the distance to the vehicle in front, and the system automatically slows down to keep a safe gap when that vehicle slows down, then speeds up again to the chosen speed when the road ahead is clear [7].

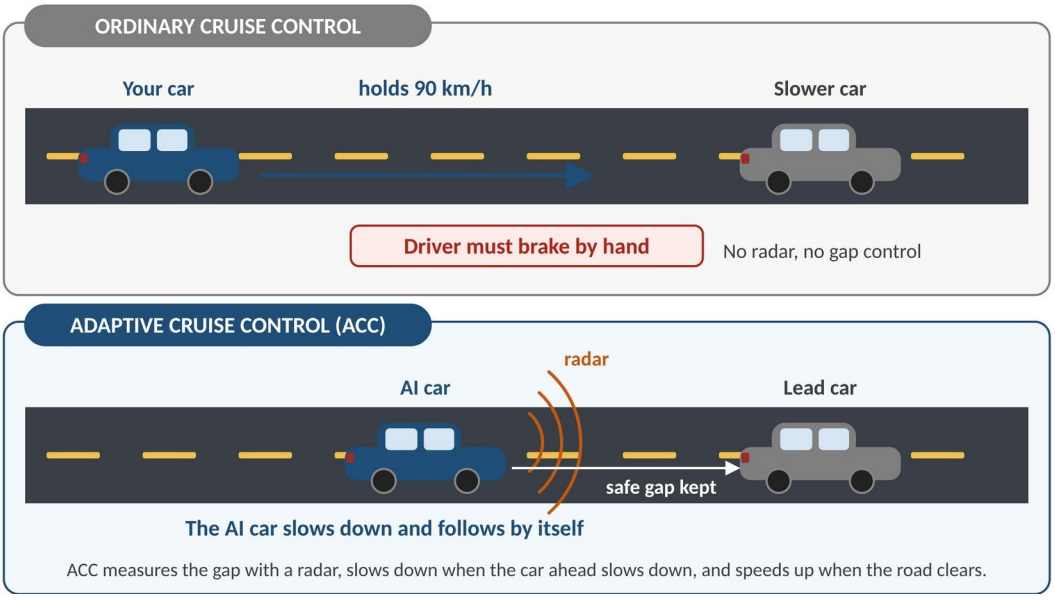


Figure 1.1: ORDINARY CRUISE CONTROL AND ACC

Most ACC systems in use today are built from **fixed control rules**, such as the constant time-gap policy [8], in which the desired gap grows with the speed of the car. These rules work well in normal traffic, but engineers must design and tune them by hand, and a single set of rules can react too late in unusual situations, for example when a car far ahead suddenly brakes hard to a stop.

This project takes a different approach. Instead of writing the driving rules, we let an **Artificial Intelligence (AI) driver learn them by itself** using **Reinforcement Learning (RL)** [4]. The idea is the same as a learner driver practising with an instructor: the AI tries different pedal actions on a simulated road, receives marks for good driving (a safe gap, a steady speed, smooth pedals) and loses marks for bad driving (too close, harsh braking, speeding and, above all, a collision), and slowly improves its own decisions over thousands of practice drives.

The whole system is **built from scratch in plain MATLAB**. The simulated road, the neural network and its backpropagation [5], the Adam optimiser [3], the experience replay buffer [6], the Double Deep Q-Network (Double DQN) learning algorithm [1], [2] and the training loop were all written by us. No pretrained model, no ready-made reinforcement learning library and no Deep Learning Toolbox is used.

What sets this project apart is its emphasis on **honest, fair testing and on learning from failure**. The AI is compared with a carefully tuned conventional controller on 105 test drives that it has never seen, with the same road, the same pedal limits and the same information. A real-time **3D live demonstration** lets a person drive the lead car and watch the AI react. This demonstration exposed a weakness that the test drives had not: the first version of the AI crashed when the car ahead suddenly stopped. Finding the cause and fixing it became an important part of the project.

The architecture also leaves room for growth. Future work can add continuous pedal control for smoother driving, sensor noise and delay, several vehicles in traffic, and testing on real driving data. With its emphasis on safety, accessibility and clear explanation, the project is not only a working AI driver but also a complete example of how a learning-based controller can be built, tested and critically evaluated.

CHAPTER 2
DESCRIPTION OF THE PROJECT

The project is a complete **learning-based Adaptive Cruise Control system** implemented in simulation. It revolves around one task: a following car (the *ego* car, driven by the AI) must follow a lead car on a straight road, keep a **safe distance** that grows with speed, hold a cruise speed of **25 m/s (90 km/h)** when the road is clear, drive smoothly, and **never collide**. The safe distance is defined as d_safe = 5 m + 1.5 s × v, which is 42.5 m at the cruise speed.

The system has six cooperating parts (Figure 2.1). The **environment** simulates the road and both cars in steps of 0.1 s. The **agent** is a neural network that looks at six numbers and chooses one of ten pedal levels. The **reward function** is the marking scheme that tells the agent how well it is doing. The **trainer** lets the agent practise for 800 simulated drives and improves its network from experience. The **evaluator** compares the trained AI with a conventional controller on unseen drives. Finally, the **3D live demo** shows the AI driving in real time while a person controls the lead car.

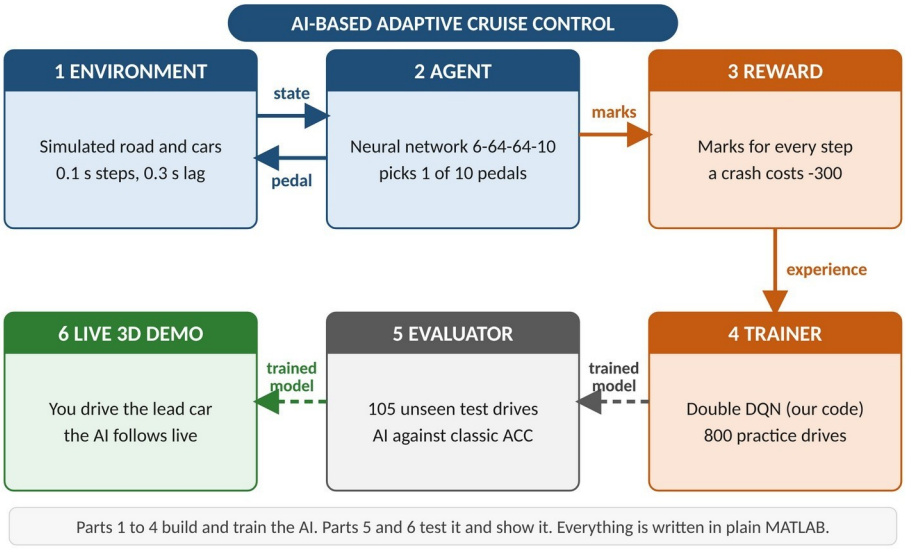


Figure 2.1: OVERALL STRUCTURE OF THE PROJECT

The conventional controller used for comparison is a **constant time-gap PD controller**, the kind of rule-based ACC used in real vehicles. It was tuned with the same marking scheme as the AI so that the comparison is fair. Both drivers face exactly the same road, the same engine and brake lag (0.3 s), the same pedal limits (-3 to +2 m/s²) and the same lead-car information.

The project places strong emphasis on **reproducibility and honesty**. Every random choice uses a fixed seed, the 105 test drives are saved to a file, and separate sets of drives are used for practice, mock testing, development and the final test.

The software is organised as small, readable MATLAB files, each with one job. This **modular design** makes it easy to change one part, such as the reward or the controller, without touching the others, and to extend the project later with continuous control, sensor noise or several vehicles.

CHAPTER 3
METHODOLOGY

The methodology follows the standard reinforcement learning cycle. At every time step of 0.1 s the **agent** (the AI driver) observes the **state** of the road, chooses an **action** (a pedal level), the **environment** moves the cars forward and returns a **reward** (marks), and the agent uses this experience to improve its decisions. The cycle is repeated for 800 practice drives of 60 s each, and the best model is then tested on drives it has never seen.

The sections below describe the tools used, the structure of the system, the simulated environment, what the agent sees and does, its neural network, the reward, the conventional controller used for comparison, and the data sets used for training and testing.

3.1. Software and Tools Used:

- **MATLAB (base, no toolboxes)**: the main platform for the simulation, the AI training, the testing and the 3D demo.
- **GNU Octave**: a free MATLAB-compatible program used for development and testing.
- **Hand-written Neural Network**: a multi-layer perceptron (6-64-64-10, ReLU) with its own forward pass and backpropagation.
- **Double DQN**: the reinforcement learning algorithm [1], [2] that teaches the network, using a target network.
- **Adam Optimiser**: updates the network weights in small, adaptive steps [3].
- **Experience Replay Buffer**: a notebook of the latest 100,000 moments from which the AI studies random samples [6].
- **Simulated Environment**: one-dimensional car physics with a 0.3 s engine and brake lag and a collision check.
- **Classic PD Controller**: a conventional constant time-gap ACC, tuned fairly and used as the benchmark.
- **MATLAB 3D Graphics**: patch and surface objects, textures and lighting for the live 3D demo.
- **Git and GitHub**: version control and teamwork.

3.2. System Architecture:

Figure 3.1 shows the structure of the system. The upper loop is the **driving loop**, which runs every 0.1 s: the lead car moves, the environment updates the physics, the six state numbers are produced, the neural network chooses one of ten pedal levels, and the chosen action moves the ego car. The lower loop is the **learning loop**, which is active only during training: every moment receives a reward, the experience is stored in a replay buffer, and the Double DQN algorithm uses stored experience to improve the network.

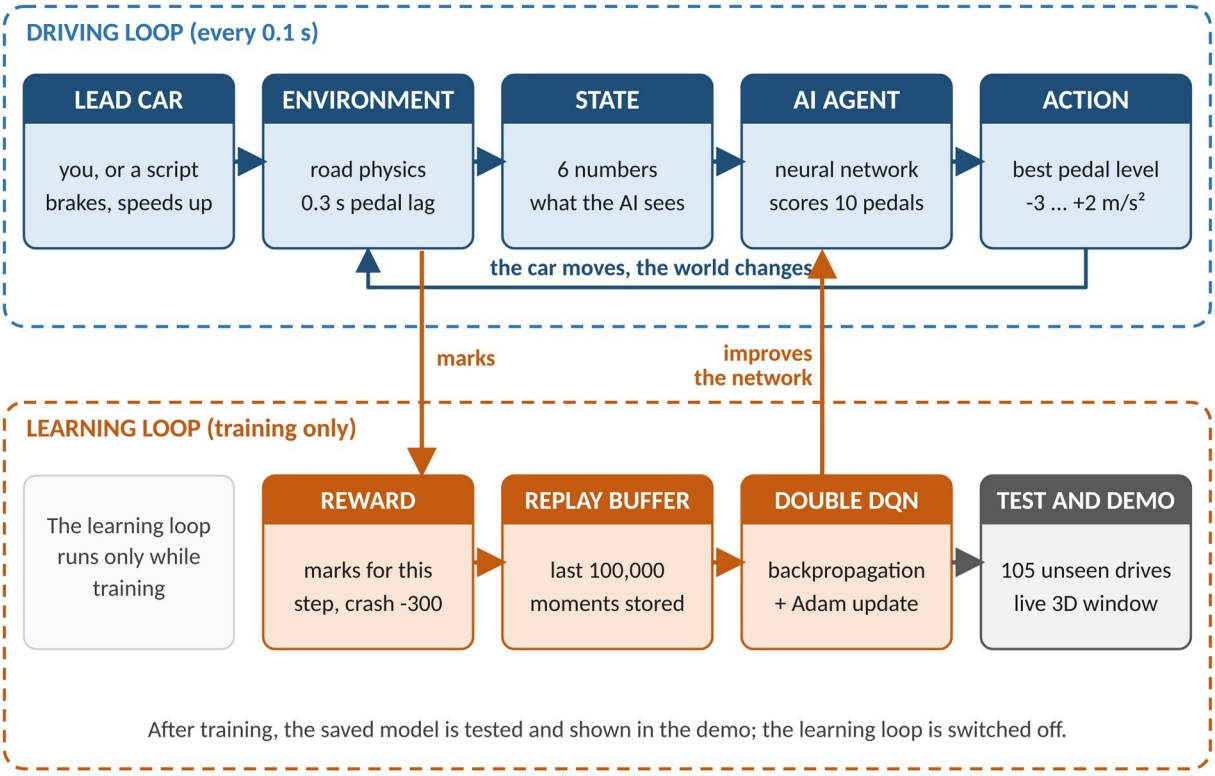


Figure 3.1: BLOCK DIAGRAM OF THE SYSTEM

3.3. Simulated Environment:

The environment is a one-dimensional straight road with two cars: the ego car (controlled by the AI or by the classic controller) and the lead car (following a script, or controlled by the user in the live demo). Every step of 0.1 s the requested acceleration of the ego car is limited to the range -3 to +2 m/s². A real engine and brake system cannot change acceleration instantly, so the actual acceleration follows the request with a **first-order lag** of 0.3 s: $a(k+1) = a(k) + (a_{cmd} - a(k)) \times \Delta t / \tau$. Speed and position are then updated with the standard equations $v(k+1) = v(k) + a \times \Delta t$ and $x(k+1) = x(k) +$ average speed $\times \Delta t$.

The gap is the lead-car position minus the ego-car position. If the gap becomes zero or negative, a **collision** is declared and the practice drive ends; a drive that survives for 60 s (600 steps) ends normally. The radar sees up to 150 m ahead, and beyond this range the road is treated as clear. The cruise speed is 25 m/s, speeds range from 0 to 35 m/s, and the safe distance is $d_{safe} = 5$ m + 1.5 s × speed (Figure 3.2).

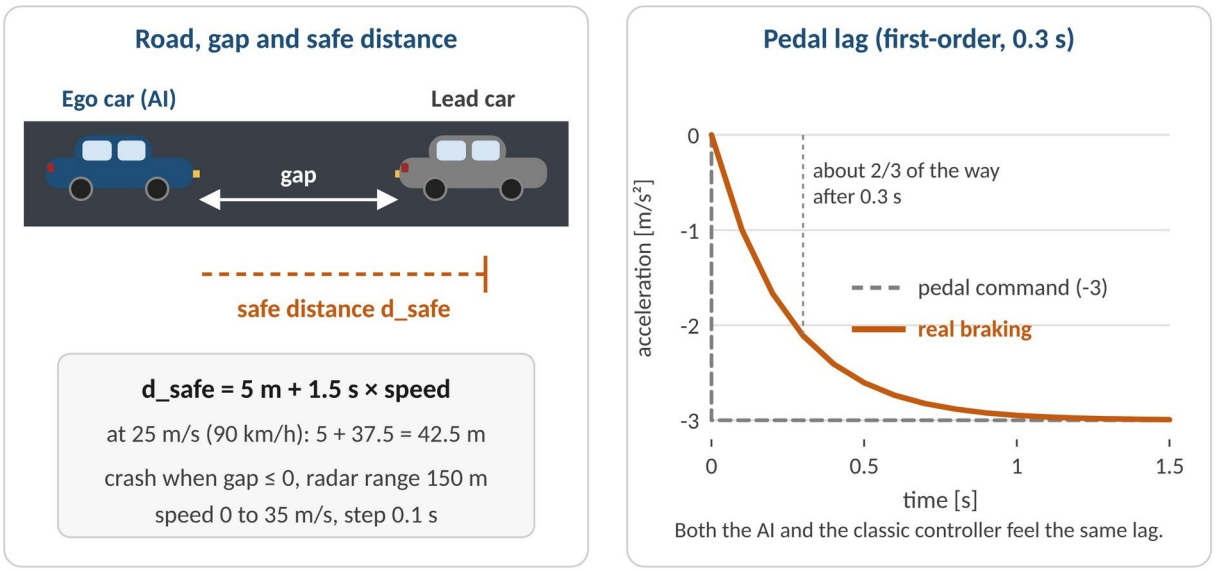


Figure 3.2: ROAD, SAFE DISTANCE AND PEDAL LAG

3.4. State and Actions:

Every 0.1 s the AI receives **six numbers**, like a driver glancing at the speedometer and the road: (1) own speed compared with the cruise speed, (2) relative speed, (3) gap to the lead car, (4) gap error (gap minus safe distance), (5) own acceleration and (6) the acceleration of the lead car. Each number is divided by a typical size (25, 10, 150, 50, 3 and 3) so that all of them lie roughly between -1 and +1, because neural networks learn poorly when one input is of the order of 100 and another of the order of 0.1. If no car is inside the radar range, the gap is reported as 150 m and the relative speed and lead acceleration as zero.

Input 6 was added late in the project. Without it, a car far ahead that has just started braking hard looks almost identical to a car that is cruising, so the AI reacted too late in sudden-stop situations (Section 5.5). It plays the role of the brake lights of the car in front. The AI controls only the pedals: at every step it chooses one of **ten fixed acceleration levels** (-3, -2, -1, -0.5, -0.25, 0, +0.25, +0.5, +1 and +2 m/s²), where negative values mean braking (Figure 3.3).

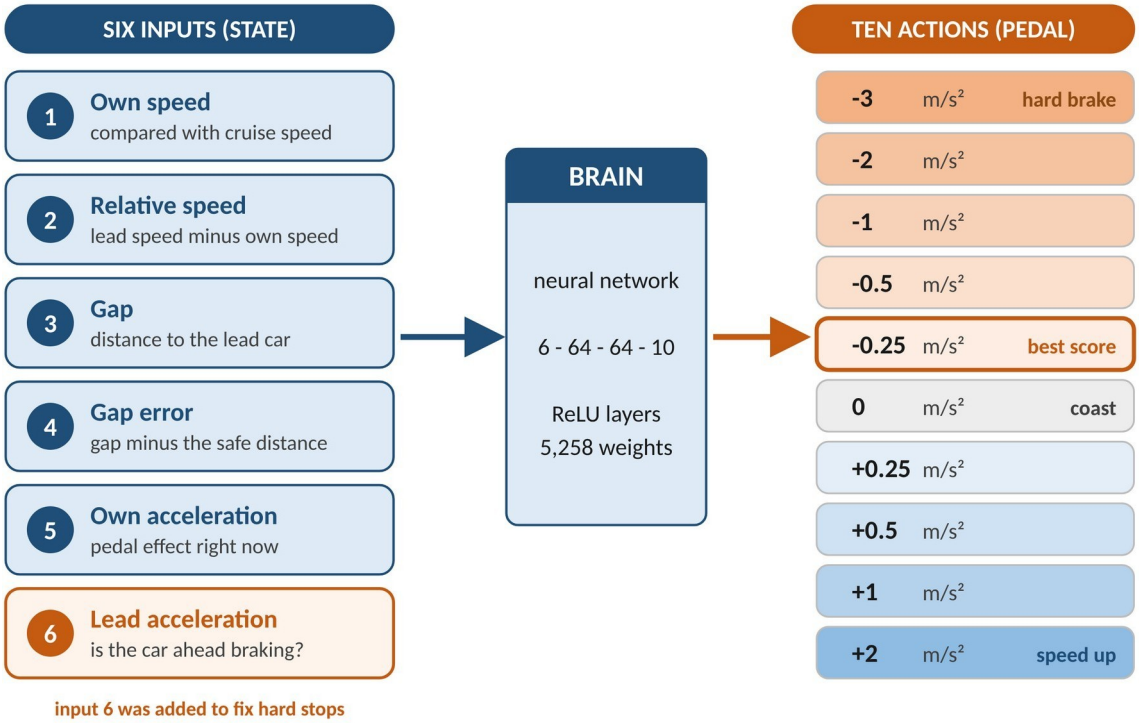


Figure 3.3: THE SIX INPUTS AND THE TEN PEDAL LEVELS

3.5. The Neural Network:

The brain of the AI is a **multi-layer perceptron** with the structure 6 - 64 - 64 - 10 (Figure 3.4). The six state numbers enter the input layer, pass through two hidden layers of 64 neurons each using the ReLU activation (a neuron outputs its input if it is positive and zero otherwise), and leave through ten linear output neurons. The network has 6×64 + 64 + 64×64 + 64 + 64×10 + 10 = **5,258 adjustable weights and biases**, all initialised randomly with the He method.

Each of the ten outputs is a **Q-value**, the network's estimate of the total future marks that will be collected if that pedal level is pressed now and the AI keeps driving well afterwards. Future marks are discounted by a factor $\gamma = 0.99$ per step, so the AI effectively looks about ten seconds ahead. To drive, the AI simply presses the pedal level with the highest Q-value. The forward pass and the backpropagation (the chain-rule calculation of how each weight affects the error) were written by hand and verified with a **gradient check**, which compares every analytic gradient with a numerical one; the relative error is about 5×10^{-12}.

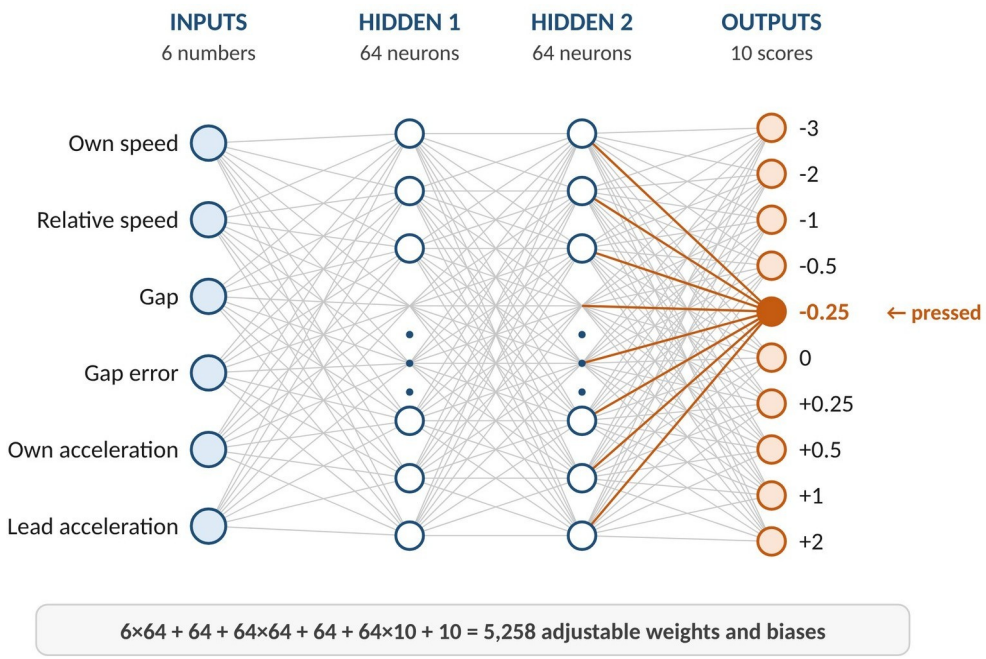


Figure 3.4: THE NEURAL NETWORK

3.6. Reward Design:

Nobody tells the AI which pedal is correct. It receives only **marks (the reward)** at every step, and everything it learns comes from them. The marking scheme (Table 3.1) rewards a gap close to the safe distance and the correct speed, and penalises danger, harsh pedal use, speeding and collisions [10]. When a car is within the radar range and is either slower than the cruise speed or closer than the safe distance, the AI is in **following mode** and is marked mostly on the gap (70%) and partly on speed (30%); on a free road it is marked on speed alone. Figure 3.5 shows the marks against the gap.

Table 3.1: The marking scheme (marks per step of 0.1 s)

Component	**Marks**
Gap close to the safe distance (following)	up to +0.7
Speed close to the correct speed	up to +0.3 (up to +1 on a free road)
Closer than the safe distance	up to -2
Closer than half the safe distance	up to -3 more
Time to collision shorter than 4 s	up to -1
Braking needed above 1.5 m/s²	up to -3
The four safety penalties above, together	limited to -2.5
Strong acceleration	$-0.05 \times (a / 3)^2$
Jerky pedal changes	$-0.20 \times (\text{jerk} / 5)^2$
Speed above the cruise speed	-0.15 per m/s
Collision	**-300, and the drive ends**

The **braking needed** term comes from the school-physics relation $v^2 = 2as$: the deceleration required to stop closing in is (closing speed)² / (2 × gap). If the lead car is itself braking, the distance it still travels before stopping is added to the gap, so the warning appears early even when the gap still looks large. The safety penalties are **capped together at 2.5 per step** and the collision penalty is -300. This was necessary because in an earlier version the small penalties of a long close call added up to more than the collision penalty, so the AI learned that crashing quickly was cheaper than suffering, and it gave up.

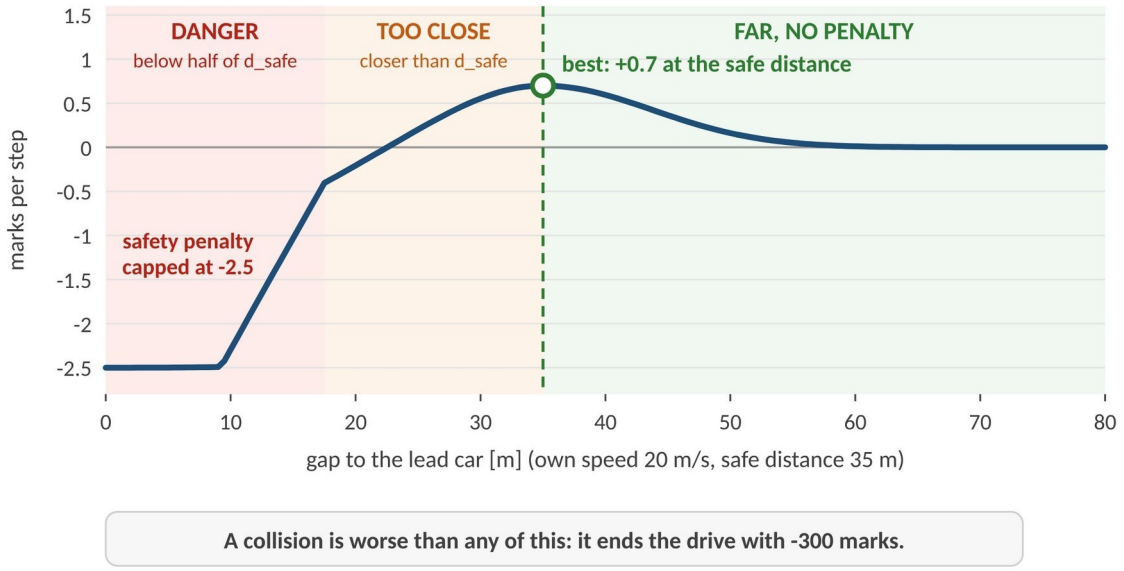


Figure 3.5: MARKS AGAINST THE GAP TO THE LEAD CAR

3.7. Classic Baseline Controller:

To judge the AI, it is compared with a conventional **constant time-gap PD controller** [8], similar to the rule-based ACC used in vehicles (Figure 3.6). It computes the acceleration command from these rules:

$$e = \min(gap - d_{safe}, e_{max})$$

$$a_{follow} = k_p \times e + k_d \times (v_{lead} - v) + k_a \times a_{lead}$$

$$a_{cruise} = k_v \times (25 - v)$$

$$a = \min(a_{cruise}, a_{follow}), \text{ limited to the range } -3 \text{ to } +2 \text{ m/s}^2$$

In words: if the car is too close or closing in, it brakes; if the lead car brakes, it brakes with it; if it is below the cruise speed and the road is clear, it accelerates; and it always follows the more careful of the two rules. The spacing error is capped at e_max because, without the cap, a large gap can cancel a fast closing speed and the controller brakes too late. The five settings were found by a two-stage **grid search** (a coarse search followed by a fine search, 594 combinations in total) on training-style drives, scored with the same marks as the AI, and any setting that crashed was rejected. The final values are k_p = 0.36, k_d = 0.56, k_v = 1.5625, e_max = 20 m and k_a = 0.30. Tuning the opponent carefully is essential: a weak opponent would make the AI look good without proving anything.

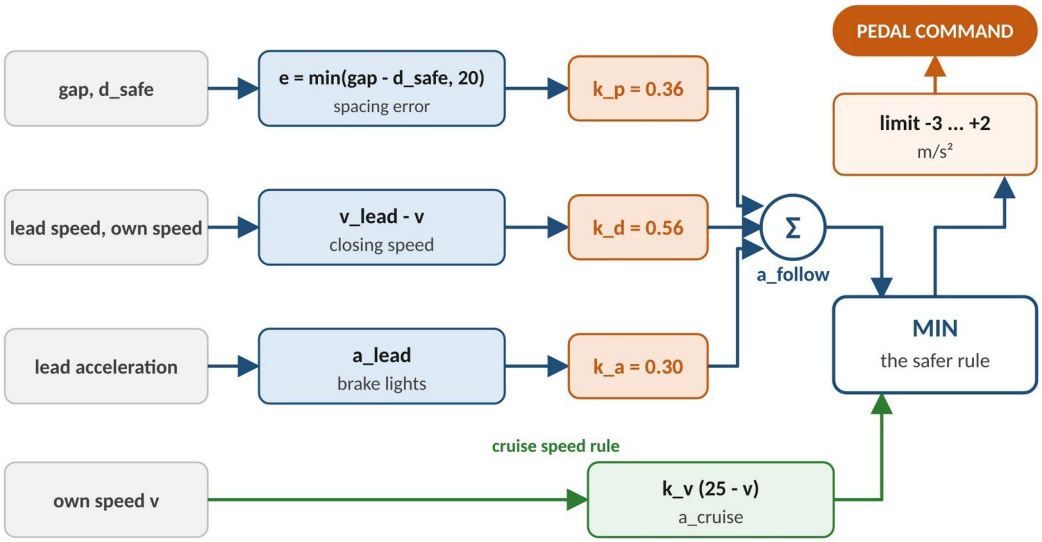


Figure 3.6: STRUCTURE OF THE CLASSIC CONTROLLER

3.8. Data Sets for Training and Testing:

Like a student, the AI must be tested on questions it has never seen. Four separate sets of drives, each created with its own random seed, were therefore used (Figure 3.7). In every drive the lead car follows a script, and eight scenario families are used: **constant** speed, **stop_and_go** traffic, **hard_brake**, **slow_then_fast**, **cut_in** (a new car suddenly enters the gap), **random_smooth** speed changes, **live_like** (imitates a person on the demo slider, practice only) and **mixed** (a random drive with a cut-in and a hard brake, final test only). The 105 final test drives are saved in a file, so the results can be reproduced exactly.

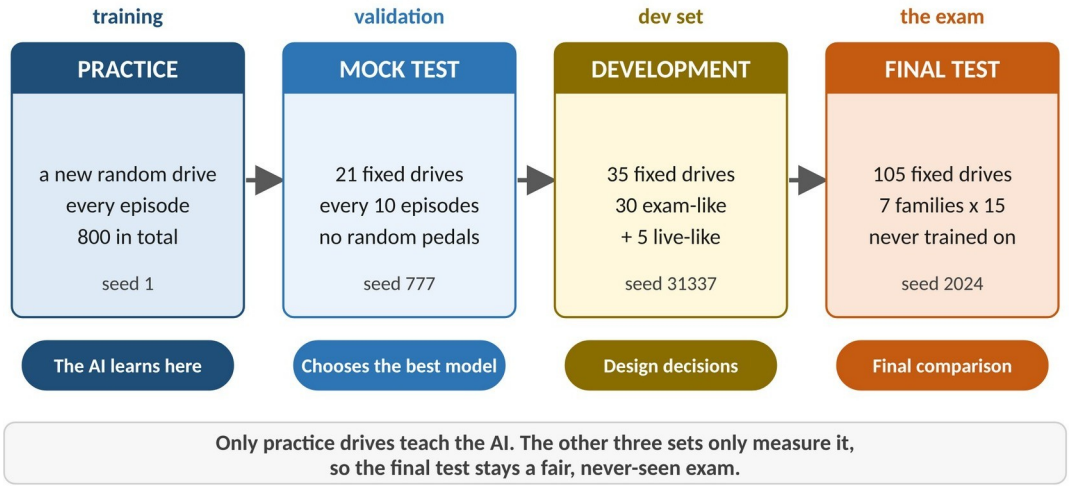


Figure 3.7: THE FOUR SETS OF DRIVES

CHAPTER 4
ALGORITHM

1. **System Initialization:**

   All settings are loaded from one configuration file (time step, limits, safe distance, reward weights and seeds). The neural network is created with random weights, a second copy of the network (the target network) is made, the Adam optimiser is reset, and an empty replay buffer of 100,000 places is prepared. The random-number generator is set to a fixed seed so that training can be repeated.

2. **Start a Practice Drive:**

   A new random scenario is generated (one of the training families, with dangerous families such as hard braking and cut-ins chosen more often) and the cars are placed at the start. A drive lasts up to 600 steps of 0.1 s.

3. **Observe the State:**

   At each step the six state numbers are computed from the speeds, accelerations and gap of the two cars.

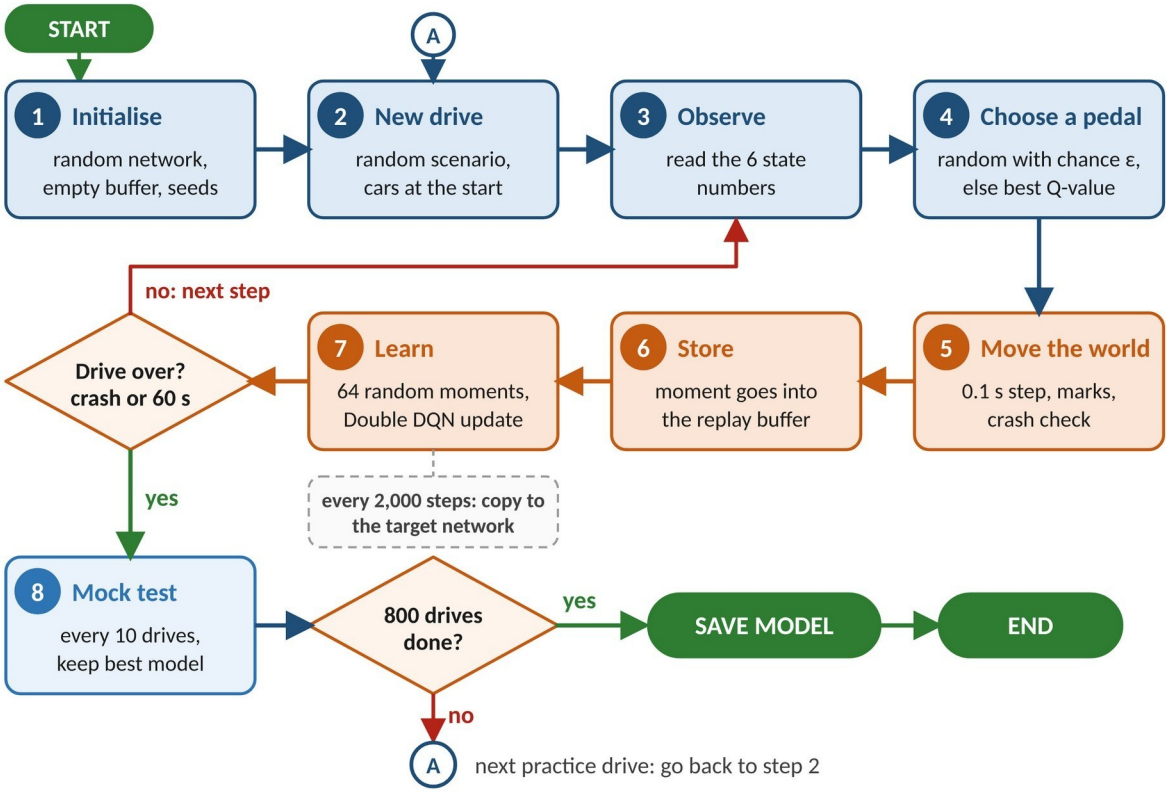


Figure 4.1: TRAINING FLOWCHART

4. **Choose an Action (Epsilon-Greedy):**

   With probability ε the AI presses a random pedal level (exploration); otherwise it presses the level with the highest Q-value (exploitation). ε starts at 1.0, which means fully random, and shrinks linearly to 0.05 over the first 480 drives.

5. **Move the World and Collect the Marks:**

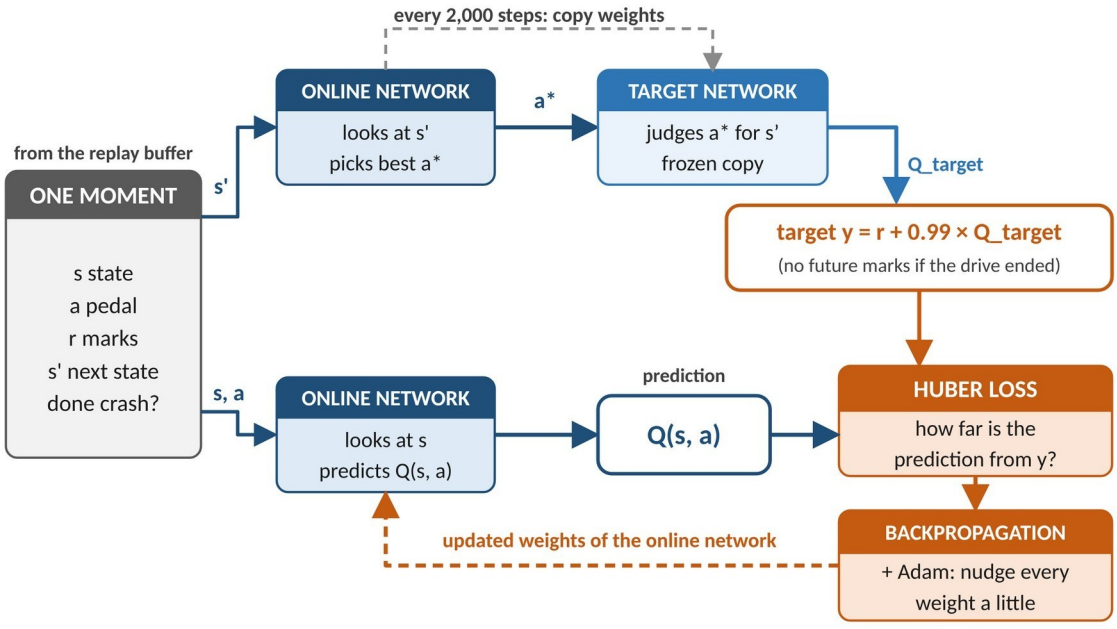
   The environment applies the pedal level with the 0.3 s lag, moves both cars by 0.1 s, checks for a collision and returns the reward. A collision ends the drive.

6. **Store the Experience:**

   The moment (state, action, reward, next state, done) is written into the replay buffer, which keeps the latest 100,000 moments.

7. **Learn from Past Experience (Double DQN):**

   After the first 3,000 steps, and after every step, the AI picks 64 random moments from the buffer. For each moment it forms a target $y = r + 0.99 \times Q_target(s', argmax_a\ Q_online(s', a))$, where the online network chooses the best next action and the target network judges it (Figure 4.2). This is the Double DQN idea, which reduces over-optimistic estimates [2]. The prediction error is measured with the Huber loss [9], backpropagation computes how every weight contributed to the error, the gradient is clipped to a maximum norm of 10, and the Adam optimiser [3] changes each weight slightly. The learning rate falls from 5×10^{-4} to 1×10^{-4} during training.



Double DQN: the online network CHOOSES the action, the target network JUDGES it.

Figure 4.2: THE DOUBLE DQN LEARNING STEP

8. **Update the Target Network:**

   Every 2,000 steps the target network is replaced by a copy of the online network. Keeping the target still between copies prevents the learning from chasing a constantly moving goal.

9. **Mock Test and Save the Best Model:**

   Every 10 drives the AI takes a mock test on 21 fixed drives with no random pedals. The best model that did not crash in the mock test is kept. After 800 drives, the model from drive 730 (mock-test score 428.7 marks per drive) was saved as the trained agent.

10. **Final Testing and Comparison:**

    The saved model and the classic controller drive the same 105 unseen drives. Collisions, gap error, speed error, RMS acceleration, RMS jerk and minimum gap are measured for every drive and compared (Chapter 5). The same model is shown in the live 3D demo.

Training of the final model took about 19 minutes on one CPU core in GNU Octave.

CHAPTER 5
USE CASES OF AI-BASED ACC AND WORKING PROCESS

5.1. Use Cases:

1. **Highway Driving Assistance:**

   On a highway the AI holds the cruise speed on a clear road and follows slower traffic at a safe distance, then speeds up again when the road clears, which reduces the workload of the driver on long journeys.

2. **Stop-and-Go Traffic:**

   In traffic jams the lead car repeatedly slows, stops and moves on. The AI follows it without the driver having to work the pedals constantly, which reduces fatigue.

3. **Sudden Braking and Emergency Situations:**

   When the car ahead brakes hard, the AI senses the braking through the lead-car acceleration input and begins to slow down at once. In our stress test it handled 60 hard-stop drives without a collision.

4. **Vehicles Cutting In:**

   When another vehicle moves into the gap in front, the gap suddenly becomes smaller. The AI reacts by restoring the gap as quickly as comfortable pedal use allows. The cut-in family is part of both training and testing.

5. **Teaching and Research in Reinforcement Learning:**

   Because every part is written in plain, commented MATLAB, the project can be used to teach step by step how neural networks, backpropagation and deep Q-learning work.

6. **Test Bed for Controller Design:**

   The environment, the metrics and the fixed test drives form a ready platform for comparing any new controller, rule-based or learned, with the existing ones fairly.

5.2. Working Process:

The working process begins when a drive starts. The ego car (blue) starts at 25 m/s behind a lead car (grey) that is 60 m ahead and travelling at 22 m/s. In the live 3D demo the user controls the lead car with the arrow keys, a slider or the STOP button; in a test drive the lead car follows a script. At every step of 0.1 s the simulator moves the lead car and the AI reads its six state numbers.

The trained neural network converts the six numbers into ten Q-values and the AI presses the pedal level with the highest value. The simulator applies that pedal level with the 0.3 s engine and brake lag, moves the ego car and updates the gap. If the lead car slows down, the gap error and the lead-car acceleration inputs change, and the AI chooses braking levels early and smoothly; when the lead car speeds up or the road clears, it accelerates back to the cruise speed, never above it.

The GUI is built from standard MATLAB figure, axes and uicontrol objects. Figure 5.1 shows how it is connected to the algorithm: the file **trained_agent.mat**, written once by the training code, is loaded by **demo.m**, and at every step the network is used only to choose a pedal. The demo never learns; training and the GUI share only the saved file.

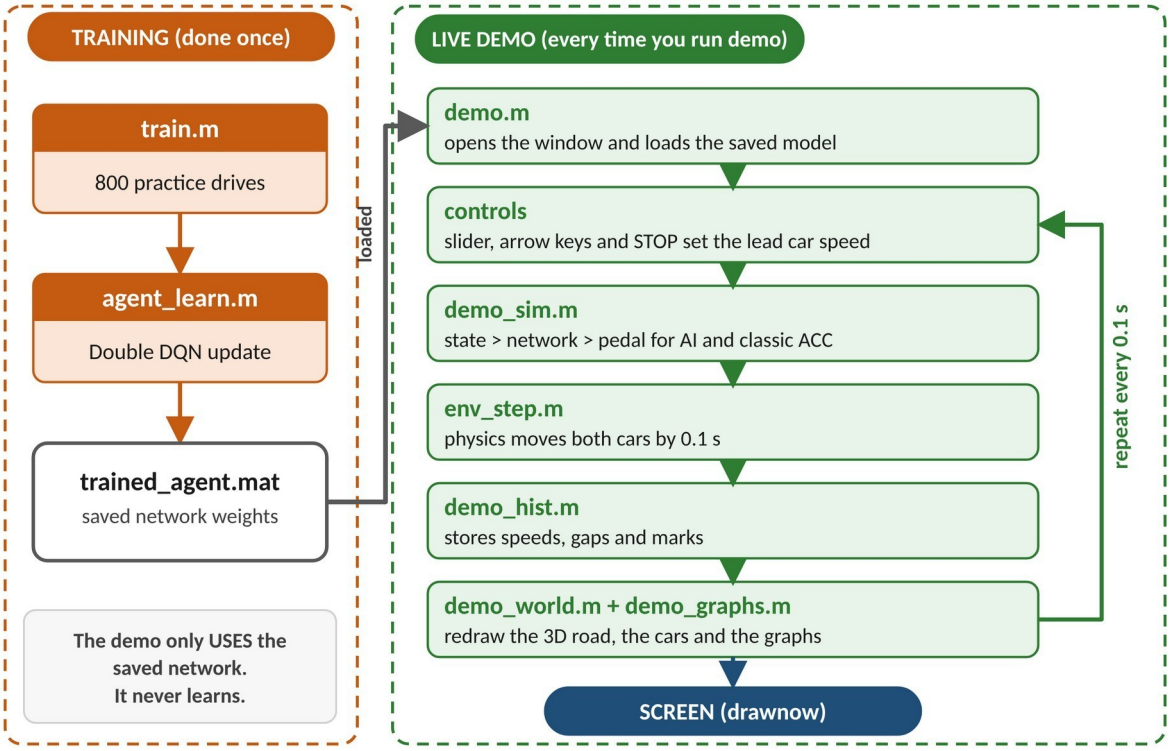


Figure 5.1: HOW THE TRAINED MODEL CONNECTS TO THE GUI

Figure 5.2 shows the layout of the live demo window. The top bar holds the speed slider and the buttons (-1 m/s, +1 m/s, STOP, Camera, Compare, Graphs, Pause and Restart; the same actions are available on the keyboard). The classic controller can be shown in the neighbouring lane behind its own copy of the lead car (Compare mode), so both drivers face the same situation side by side. A replay mode, demo('hard_brake', 1), plays back any unseen test drive.

Because a person can brake the lead car harder (4 m/s²) than ACC is allowed to brake (3 m/s²), the live demo also contains an **automatic emergency braking (AEB) layer**, as real vehicles do. It acts only when a collision is close (a required deceleration above 2.8 m/s², or a time to collision below 1.5 s), brakes at 7 m/s², shows an orange warning and counts how often each driver needed it. AEB is **never used in the final test or in the stress test**.

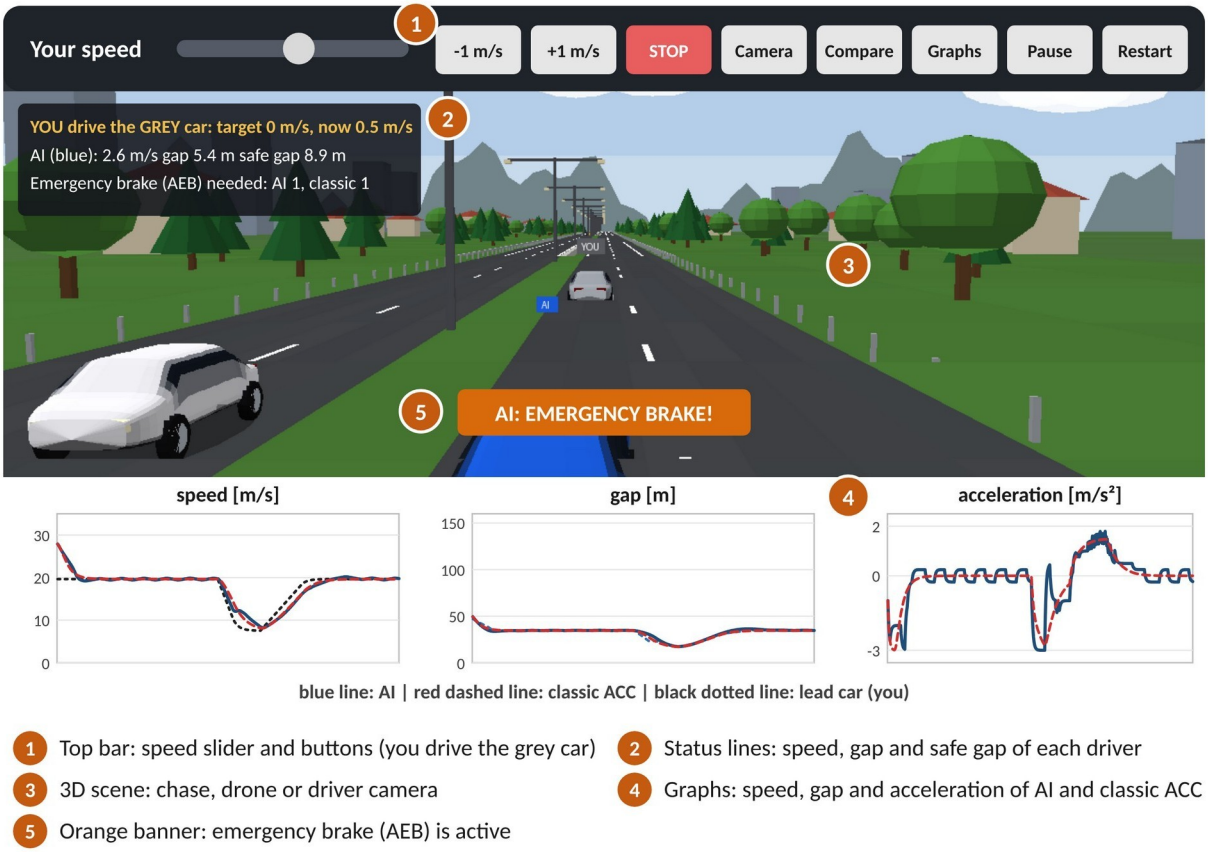


Figure 5.2: LAYOUT OF THE LIVE DEMO WINDOW

5.3. Training Result:

Figure 5.3 shows the progress of training. At the beginning the AI presses random pedals and crashes often, so the marks per drive are far below zero. As the Q-values become accurate, the marks rise steadily. The orange line is the mock test (no random pedals); it passes 400 marks per drive at drive 260 and then stays between about 385 and 430. The grey line is the average of the practice drives, which is noisier because it still includes random exploring. A perfect 60 s drive would score about 600 marks. The saved model came from drive 730, with a mock-test score of 428.7 marks per drive and no collisions.

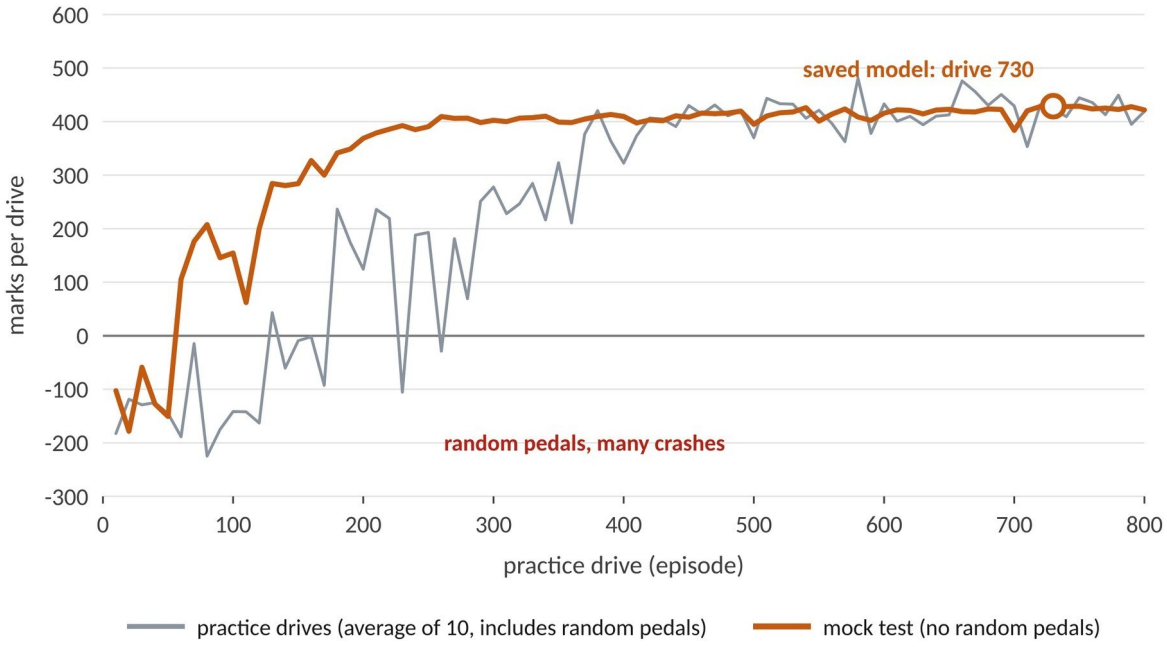


Figure 5.3: TRAINING CURVE

5.4. Test Results:

Both drivers were tested on the same 105 unseen drives (7 families × 15). Figure 5.4 shows one of them, a hard-brake drive, recorded from the real simulation: the lead car slows suddenly at about 28 s, and both drivers brake smoothly, never crash, and recover the cruise gap. Table 5.2 and Figure 5.5 summarise all 105 drives.

Table 5.2: Results on the 105 unseen test drives

Metric	AI (DQN)	Classic ACC
Collisions	0	0
Gap error [m]	10.14	6.14
Speed error RMSE [m/s]	2.60	2.61
RMS acceleration [m/s²]	0.85	0.88
RMS jerk [m/s³]	1.43	0.69
Worst minimum gap [m]	4.85	5.00
Average minimum gap [m]	20.13	19.20
Average following distance [m]	38.74	35.18
Average safe distance [m]	29.63	29.84
Average total marks per drive	447.9	473.1

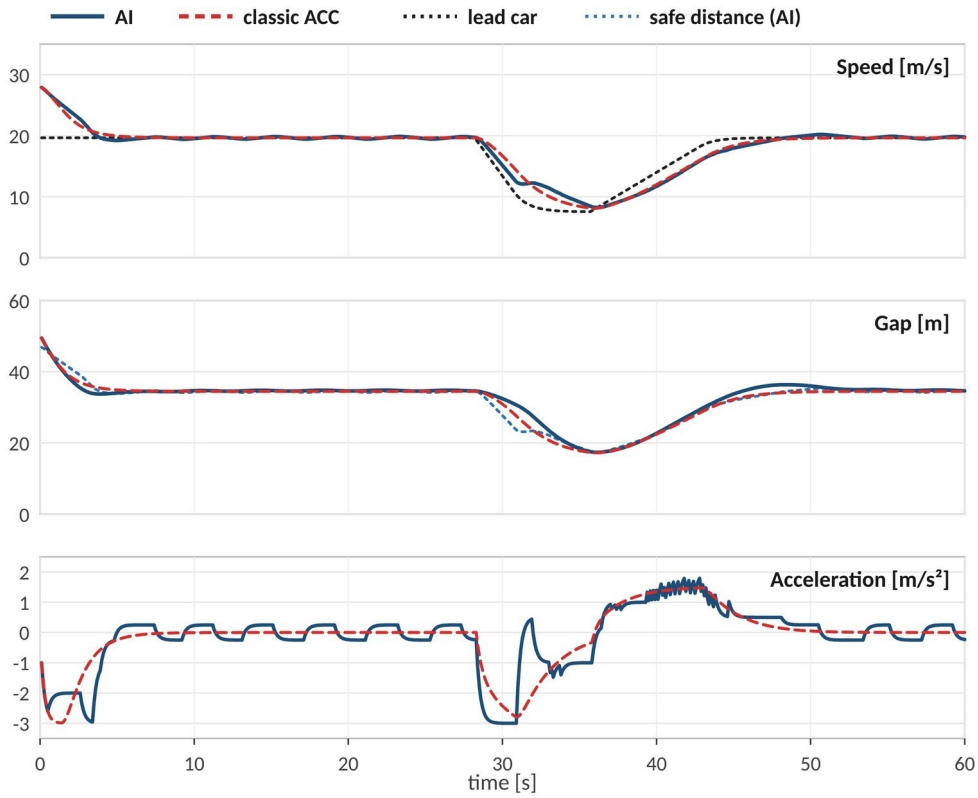


Figure 5.4: AI AND CLASSIC ACC ON A HARD-BRAKE DRIVE

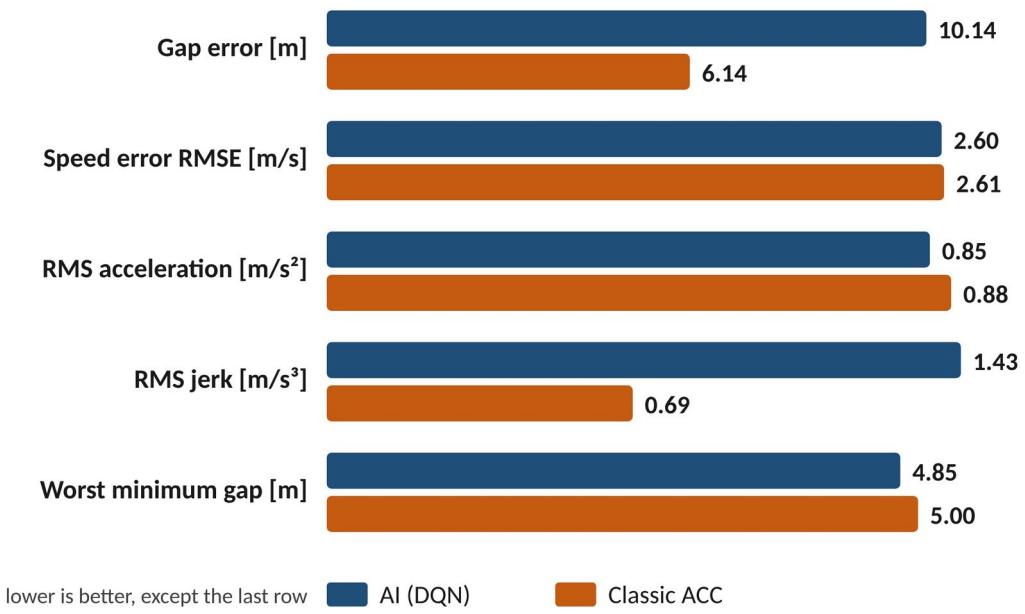


Figure 5.5: RESULTS ON 105 UNSEEN DRIVES

Gap error is the average difference between the actual gap and the safe distance while following. **Speed error (RMSE)** is the root-mean-square difference between the speed and the correct speed. **RMS acceleration** and **RMS jerk** measure how hard and how suddenly the pedals are used (comfort); the root-mean-square squares every value, averages them and takes the square root, so large values count more.

Safety. Neither driver collided in any of the 105 drives, including the family (mixed) that the AI never practised. The smallest gap reached by the AI was 4.85 m. **Speed and acceleration.** The AI ties the classic controller on speed error (2.60 against 2.61 m/s) and is slightly gentler on acceleration (0.85 against 0.88 m/s²). **Gap accuracy.** The classic controller holds the gap closer to the safe distance (a gap error of 6.14 m against 10.14 m). The AI is a cautious driver: it follows on average 38.7 m behind when the safe distance is 29.6 m, about 9 m further back than necessary. **Smoothness.** The classic controller is smoother (jerk 0.69 against 1.43 m/s³) because it can output any acceleration, while the AI chooses from ten fixed levels and moves between neighbouring levels.

Table 5.3: Results by scenario family (AI / Classic ACC)

Family	Gap error [m]	Speed RMSE [m/s]	Minimum gap [m]	Collisions
constant	8.58 / 3.25	1.74 / 1.78	16.4 / 16.4	0 / 0
cut_in	15.83 / 11.94	2.00 / 2.11	15.9 / 16.0	0 / 0
hard_brake	15.45 / 7.40	2.49 / 2.34	12.8 / 11.8	0 / 0
mixed	7.49 / 5.87	2.77 / 2.92	4.9 / 5.0	0 / 0
random_smooth	13.49 / 7.17	3.26 / 3.22	14.7 / 14.7	0 / 0
slow_then_fast	5.54 / 3.96	3.17 / 3.15	17.5 / 17.7	0 / 0
stop_and_go	4.45 / 3.19	2.80 / 2.78	6.0 / 6.6	0 / 0

Transparency of testing. The 105-drive test was run four times in total during the project. Every design decision was taken on a separate development set, and the final model was chosen on that set before the final run. It was kept even though the final run showed a larger gap error than an earlier model, because changing the model after seeing the test result would amount to choosing the answer from the test paper.

5.5. Hard-Stop Safety Test:

While driving the lead car by hand in the live demo, we found a situation that none of the test drives contained: the car ahead braking hard to a full stop. The first version of the AI crashed. Three causes were found and fixed. First, the AI had learned to **give up**: near a crash the small safety penalties added up to more than the collision penalty, so the collision penalty was raised to -300 and the safety penalties were capped. Second, the AI sometimes **exceeded the cruise speed**, so a penalty for speeding was added. Third, and most important, the AI **could not see that the car ahead was braking**, because a car 100 m ahead that has just started braking looks almost the same as a cruising car. The lead-car acceleration was therefore added as the sixth input, the reward was made to account for the lead car's stopping distance, and the classic controller was given the same information as a feed-forward term to keep the comparison fair.

To measure the effect, a stress test of **60 hard-stop drives** was built with the emergency brake switched off. The lead car cruises at 12, 18, 22, 26 or 30 m/s; at 8, 20 or 40 s it brakes at 3 or 4 m/s² to a stop (0 m/s) or to 8 m/s; and later it speeds up again (5 × 3 × 2 × 2 = 60 drives). The

earlier AI crashed in 7 of them and the classic controller in 8, while the final AI crashed in none and never came closer than 4.7 m (Figure 5.6).

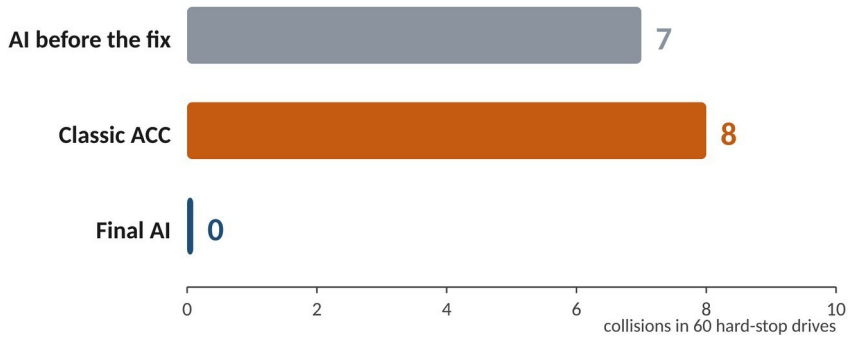


Figure 5.6: COLLISIONS IN THE HARD-STOP TEST

All eight collisions of the classic controller occurred when the lead car was cruising faster (26 or 30 m/s) than the cruise speed of 25 m/s and then stopped completely: the gap had grown large, the formula still felt safe, and by the time the closing speed became large a deceleration of 3 m/s² was no longer enough. The final AI, which sees the lead car braking, started slowing early. This stress test was designed after the weakness was found, so it is reported separately from the 105-drive final test.

CHAPTER 6
FEATURES OF THIS PROJECT

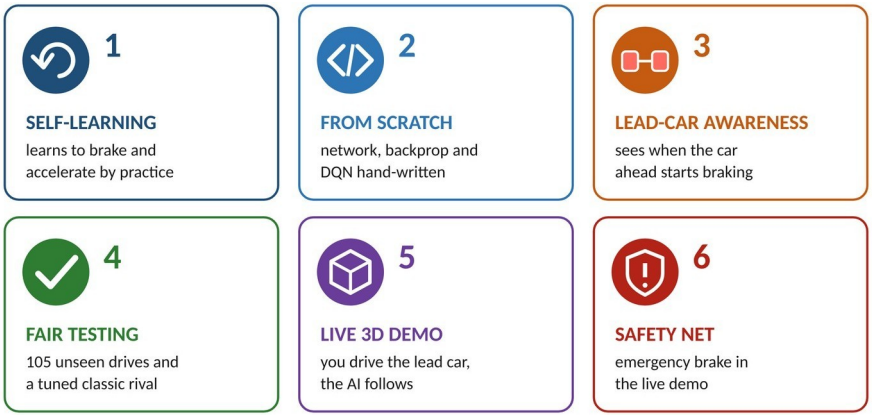


Figure 6.1: KEY FEATURES AT A GLANCE

1. **Self-Learning Driver**

   The AI is not given driving rules. It learns when to brake and when to accelerate by practising 800 simulated drives and collecting marks, so the same code can be retrained for a different road or reward.

2. **Built From Scratch**

   The road, the neural network, backpropagation, the Adam optimiser, the replay buffer, the Double DQN learning step and the training loop are all hand-written in plain MATLAB, with no pretrained model, toolbox or library. Backpropagation is checked against numerical gradients.

3. **Lead-Car Awareness (Brake-Light Input)**

   The AI sees not only the gap and the speeds but also the acceleration of the lead car, which lets it react to braking early. This single extra input reduced the collisions in the 60-drive hard-stop test from 7 to 0.

4. **Fair and Honest Testing**

   The AI is compared with a carefully tuned classic controller under identical conditions, on 105 drives it has never seen, with fixed seeds and saved test drives so that results can be repeated. Weaknesses are reported together with strengths.

5. **Interactive 3D Live Demo**

   A person drives the lead car with the keyboard, a slider or a STOP button and watches the AI react in real time from a chase, drone or driver’s-seat camera. A Compare mode shows the classic controller side by side, and live graphs show speed, gap and acceleration.

6. **Emergency Braking Safety Net**

   Like real vehicles, the live demo adds an automatic emergency braking layer that acts only when a collision is close and records how often each driver needed it. It is used only in the demonstration and never in the testing, so the test results show the AI’s own ability.

CHAPTER 7
CONCLUSION

The development of the **AI-Based Adaptive Cruise Control Using Reinforcement Learning** shows that a driving controller can be learned from experience instead of being programmed. Starting from random pedal choices, the AI learned over 800 practice drives, using only a marking scheme, to follow a lead car at a safe distance, hold the cruise speed and brake smoothly. It recorded **zero collisions in 105 unseen test drives**, matched a carefully tuned conventional controller on speed tracking and acceleration, and recorded **zero collisions against eight** for the conventional controller in a 60-drive hard-stop stress test.

The project also shows the value of testing in situations the designers did not plan. A weakness (the AI could not see that the car ahead was braking) was not visible in the test drives but appeared as soon as a person drove the lead car in the live 3D demo. Fixing it with one extra input and a better marking scheme was as important as the learning algorithm itself.

The results are reported honestly. The final AI is a cautious driver that keeps about 9 m more than the safe distance, so its gap error (10.1 m) is larger than that of the classic controller (6.1 m), and its pedal use is less smooth (RMS jerk 1.43 against 0.69 m/s³) because it chooses from ten fixed acceleration levels. **Future work** includes continuous-action methods such as DDPG or TD3 for smoother driving, a stronger gap reward, sensor noise and delay, several vehicles in traffic, and testing with real driving data.

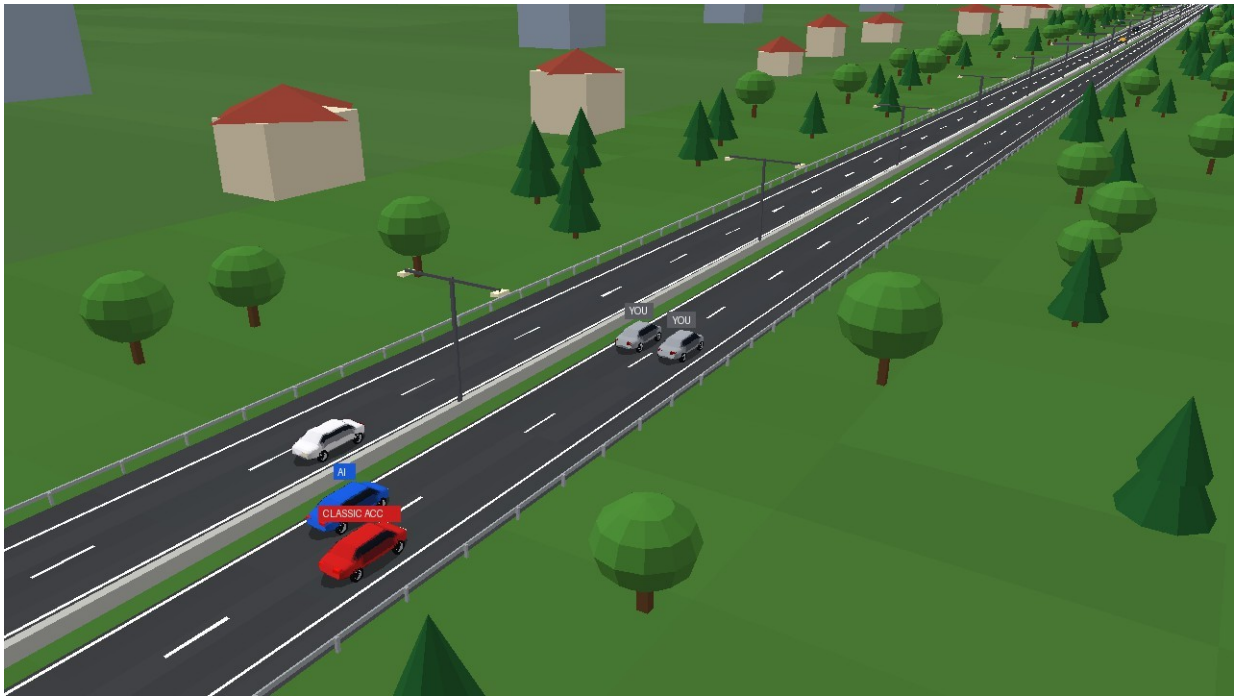


Figure 7.1: FINAL PRODUCT: THE LIVE 3D DEMO

CHAPTER 8
REFERENCES

[1] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529-533, 2015. [DOI:10.1038/nature14236]

[2] H. van Hasselt, A. Guez and D. Silver, "Deep Reinforcement Learning with Double Q-learning," in *Proc. 30th AAAI Conference on Artificial Intelligence*, Phoenix, AZ, USA, 2016, pp. 2094-2100.

[3] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. 3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015. [arXiv:1412.6980]

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[5] D. E. Rumelhart, G. E. Hinton and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533-536, 1986.

[6] L.-J. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," *Machine Learning*, vol. 8, no. 3-4, pp. 293-321, 1992.

[7] ISO 15622:2018, "Intelligent transport systems - Adaptive cruise control systems - Performance requirements and test procedures," International Organization for Standardization, Geneva, Switzerland, 2018.

[8] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York, NY, USA: Springer, 2012.

[9] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73-101, 1964.

[10] A. Y. Ng, D. Harada and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. 16th International Conference on Machine Learning (ICML)*, Bled, Slovenia, 1999, pp. 278-287.